Reproducing HybridSN: Validation, PCA Leakage, and Spatial Block Splits

Chaknith Bin

***Abstract*—This paper presents a comprehensive scientific verification of the Hybrid Spectral Convolutional Neural Network (HybridSN) [1]. While the original paper demonstrates state-of-the-art performance on Indian Pines (IP), University of Pavia (UP), and Salinas Scene (SA) datasets, this research extends the original evaluation by examining sensitivity to PCA settings, the necessity of validation sets for model selection, and the effect of spatial data leakage. Additionally, the research evaluates the model's generalization capability through block-based spatial splitting, showing that performance estimates are highly sensitive to the sampling protocol. The source code can be found at https://github.com/Chaknith/Recreate-Research-Paper.**

***Index Terms*—Hyperspectral image classification, remote sensing, HybridSN, convolutional neural networks, principal component analysis, data leakage, spatially disjoint evaluation, block splitting, reproducibility.**

I. Introduction

Hyperspectral image (HSI) classification requires capturing both spectral and spatial correlations between bands. The HybridSN model was proposed to bridge the gap between computationally expensive 3D-CNNs and spatially focused 2D-CNNs. Although the original study reported near-perfect classification accuracy, the effects of alternative benchmarking practices, such as train-only PCA and spatially disjoint splitting, remain underexplored. This study reproduces HybridSN to assess its reproducibility and evaluates it under a range of testing protocols to identify potential methodological biases in standard HSI benchmarking.

II. Background and Related Work

HSI classification aims to assign each labelled pixel to a land-cover or material class using both spectral and spatial information. Compared with RGB images, HSIs contain hundreds of narrow spectral bands, which provide rich material information but also introduce high dimensionality, limited labelled samples, and strong spatial correlation between neighbouring pixels.

Deep learning methods, especially convolutional neural networks (CNNs), have become widely used for HSI classification because they can automatically learn spectral and spatial features from image patches. HybridSN combines 3D and 2D convolutions [1]. Its 3D-CNN layers first extract joint spectral-spatial features, while later 2D-CNN layers further learn spatial representations with lower computational cost. HybridSN has reported strong results on Indian Pines (IP), University of Pavia (UP), and Salinas Scene (SA) datasets[1].

However, the evaluation protocol used in HSI classification can strongly affect reported performance. Many studies use random-pixel splitting, where pixels from the same image are randomly divided into training and testing sets. This can produce overly optimistic results because nearby pixels are highly correlated and their extracted patches may overlap. Therefore, test samples may not be fully independent from training samples [2].

A stricter alternative is disjoint or spatial splitting, where training, validation, and testing samples come from non-overlapping spatial regions. This better reflects real remote-sensing scenarios, where a model must generalise to unseen areas rather than classify pixels near the training samples. For this reason, spatially disjoint evaluation is often considered a more realistic test of generalisation [3].

Another important issue is data leakage from preprocessing. If PCA, normalisation, or other transformations are fitted before the train-test split, information from validation or test pixels may influence the training data. A leakage-safe pipeline should split the data first, fit preprocessing only on the training set, and then apply the fitted transformations to validation and test sets [4].

Reproducibility is also important because HSI results can vary across random seeds, splits, preprocessing choices, and implementation details. Therefore, reliable evaluation should report multiple runs using metrics such as overall accuracy (OA), average accuracy (AA), Cohen's kappa, and their standard deviations.

III. Reproduction Setup

This study reproduces the HybridSN model on three hyperspectral image classification datasets: Indian Pines (IP), University of Pavia (UP), and Salinas Scene (SA). These datasets were selected because they also used in the original HybridSN study.

For each dataset, the hyperspectral cube was first reduced using PCA. The number of retained principal components was set to 30 for IP dataset at the baseline testing and 15 for all three dataset for the rest of the testing. After PCA, local spatial patches were extracted around labelled pixels using a

Chaknith Bin is a student at RMIT University, Melbourne, VIC, Australia.(email: chaknithg@email.com).

[1]https://www.ehu.eus/ccwintco/index.php?title=Hyperspectral_Remote_Sensing_Scenes

[2]Public PyTorch implementation used for comparison: https://github.com/Pancakerr/HybridSN.

Table I: Baseline

	IP			UP			SA		
Method	OA	AA	Kappa	OA	AA	Kappa	OA	AA	Kappa
HybridSN [1]	99.75 ± 0.10	99.63 ± 0.20	99.71 ± 0.10	99.98 ± 0.00	99.97 ± 0.00	99.98 ± 0.00	100.00 ± 0.00	100.00 ± 0.00	100.00 ± 0.00
PyTorch reimplementation[2]	99.29 ± 0.56	98.51 ± 1.40	99.19 ± 0.64	99.96 ± 0.02	99.91 ± 0.06	99.94 ± 0.03	100.00 ± 0.00	100.00 ± 0.01	100.00 ± 0.01
Reproduction	99.37 ± 0.35	98.27 ± 1.36	99.28 ± 0.40	99.97 ± 0.03	99.91 ± 0.07	99.96 ± 0.03	99.99 ± 0.01	99.99 ± 0.01	99.99 ± 0.01

Table II: Block Split

	IP			UP			SA		
Block size	OA	AA	Kappa	OA	AA	Kappa	OA	AA	Kappa
1x1	99.60 ± 0.12	99.12 ± 0.81	99.54 ± 0.14	99.96 ± 0.03	99.90 ± 0.07	99.94 ± 0.04	99.99 ± 0.01	99.98 ± 0.02	99.99 ± 0.01
3x3	92.99 ± 1.00	89.90 ± 2.73	91.96 ± 1.13	99.78 ± 0.18	99.47 ± 0.51	99.71 ± 0.24	99.95 ± 0.04	99.93 ± 0.07	99.95 ± 0.04
5x5	82.30 ± 3.36	79.12 ± 5.62	79.74 ± 3.86	99.50 ± 0.18	98.76 ± 0.52	99.33 ± 0.24	99.89 ± 0.11	99.85 ± 0.13	99.87 ± 0.12
10x10				98.88 ± 0.62	97.66 ± 1.30	98.50 ± 0.83	99.05 ± 0.40	99.34 ± 0.45	98.93 ± 0.45
15x15				97.52 ± 0.60	95.76 ± 1.12	96.68 ± 0.79	97.39 ± 0.93	97.97 ± 0.94	97.07 ± 1.04
25x25				95.79 ± 2.13	93.37 ± 3.14	94.22 ± 2.91	92.58 ± 3.01	94.15 ± 2.82	91.61 ± 3.41
35x35				91.64 ± 2.93	88.77 ± 3.44	88.51 ± 4.01	87.84 ± 4.64	91.74 ± 3.85	86.31 ± 5.17
50x50				85.24 ± 10.55	85.71 ± 5.90	80.33 ± 12.84	78.84 ± 5.64	85.89 ± 2.68	75.45 ± 6.75
65x65				78.71 ± 7.49	83.28 ± 5.50	72.57 ± 8.96			
80x80				78.56 ± 12.71	80.86 ± 8.38	70.74 ± 14.94			

Table III: Anomalous on IP dataset

Method	PCA	OA (%)	AA (%)	Kappa
PyTorch reimplementation	30	24.48	6.25	0.00
Reproduction	50	23.97	6.25	0.00

patch size of 25×25. Background pixels were excluded from training and evaluation.

The reproduced model follows the HybridSN architecture, which combines 3D and 2D-CNN layers. The first part of the network uses 3D-CNNs to extract joint spectral-spatial features from the HSI cube. This is followed by a 2D-CNN layers and fully connected layers for final classification. The model uses three 3D-CNN layers with kernel sizes $(7, 3, 3)$, $(5, 3, 3)$, and $(3, 3, 3)$, followed by a 2D-CNN layers with kernel size $(3, 3)$. Dropout was applied before the final classification layers.

The model was trained using the Adam optimizer with a learning rate of 0.001, a batch size of 256, and a maximum of 100 training epochs. For each dataset, the labelled samples were divided into training, validation, and test sets using a $0.3/0.1/0.6$ ratio. The validation set was used for model selection, and the model checkpoint with the best validation performance was used for final testing.

To account for randomness in data splitting and model training, all experiments were repeated using ten seeds: `[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]`. Final results are reported as the mean and standard deviation across these runs.

Model performance was evaluated using three standard metrics: overall accuracy (OA), average accuracy (AA), and Cohen's kappa coefficient. OA measures the proportion of correctly classified test samples, AA measures the mean per-class accuracy, and kappa measures classification agreement while accounting for chance agreement.

All experiments were conducted on a computer equipped with an AMD Ryzen 7 5700X, an NVIDIA GeForce RTX 3090 Ti with 24 GB memory, and 32 GB RAM. The experiments were run in Ubuntu 22.04 via Windows Subsystem for Linux (WSL). The software environment used Python 3.10.12, PyTorch 2.7.1, CUDA 11.8, scikit-learn 1.7.2, and NumPy 2.2.6.

IV. Results and Discussion

A. Baseline

The accuracy scores of the HybridSN, the PyTorch reimplementation, and this reproduction are broadly comparable across the evaluated datasets, as shown in Table I. This provides initial evidence that the HybridSN architecture can be reproduced under the experimental settings used in this study, and that its performance is not solely dependent on a specific implementation. However, one result from the PyTorch reimplementation benchmark was excluded from the aggregated comparison due to anomalous behavior. This result is reported separately in Table III and discussed further in the Future Work section.

B. Validation Set

Running the benchmark without a validation set introduces the risk of selecting an overfitted model checkpoint. For example, under seed 5 with 30 PCA components, the IP result dropped to OA = 90.38, AA = 77.92, and Kappa = 89.03. This performance is noticeably lower than the other baseline results, suggesting that validation-based checkpoint selection is important for obtaining stable benchmark performance.

C. PCA and Patch Size Sensitivity

Overall, HybridSN achieved performance broadly comparable to the baseline results across different PCA component settings and patch sizes. However, one result from the PCA 50 experiment was excluded from the aggregation due to anomalous behavior. This result showed a pattern similar to the anomaly observed in the PyTorch reimplementation with PCA 30, as shown in Table III.

D. Random Split vs. Block Split

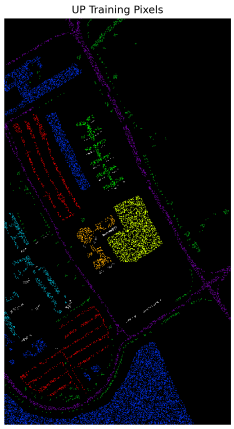
The most significant finding in this study is the performance gap between the random pixel split and the block split evaluation. This gap was measured by using the 1-pixel block result as the control score and subtracting the

Table IV: Train-Only PCA with Block Split

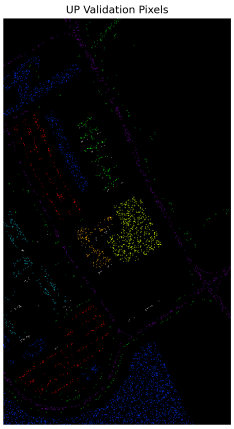
	IP			UP			SA		
Block size	OA	AA	Kappa	OA	AA	Kappa	OA	AA	Kappa
1x1	99.63 ± 0.09	98.72 ± 1.06	99.57 ± 0.11	99.95 ± 0.04	99.90 ± 0.08	99.94 ± 0.05	99.99 ± 0.02	99.98 ± 0.03	99.99 ± 0.02
3x3	93.36 ± 1.01	89.96 ± 3.20	92.39 ± 1.14	99.82 ± 0.10	99.63 ± 0.22	99.76 ± 0.13	99.98 ± 0.02	99.96 ± 0.03	99.97 ± 0.02
5x5	84.44 ± 2.28	80.07 ± 3.62	82.17 ± 2.60	99.66 ± 0.16	99.29 ± 0.35	99.54 ± 0.21	99.90 ± 0.05	99.87 ± 0.13	99.89 ± 0.06
10x10				98.65 ± 0.50	97.75 ± 1.10	98.19 ± 0.67	99.31 ± 0.39	99.52 ± 0.31	99.23 ± 0.44
15x15				97.65 ± 0.41	95.82 ± 0.85	96.85 ± 0.56	98.13 ± 1.03	98.68 ± 0.82	97.90 ± 1.16
25x25				95.70 ± 1.09	94.16 ± 0.85	94.09 ± 1.45	92.62 ± 2.45	94.45 ± 3.45	91.66 ± 2.73
35x35				92.41 ± 2.79	90.31 ± 3.21	89.64 ± 3.66	87.88 ± 4.41	92.49 ± 3.62	86.37 ± 4.95
50x50				88.76 ± 5.18	87.24 ± 5.42	84.59 ± 7.22	80.08 ± 5.33	86.40 ± 3.19	76.85 ± 6.19
65x65				80.78 ± 7.61	83.39 ± 4.64	75.09 ± 8.25			
80x80				79.67 ± 12.75	81.49 ± 8.22	72.19 ± 14.42			



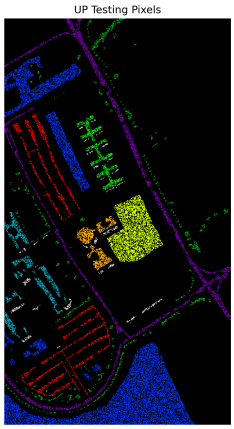
(a) UP Block 1x1: overview



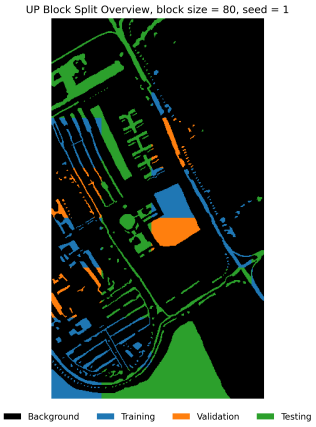
(b) UP Block 1x1: training



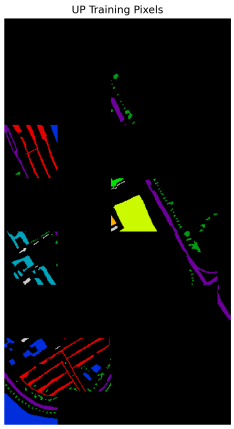
(c) UP Block 1x1: validation



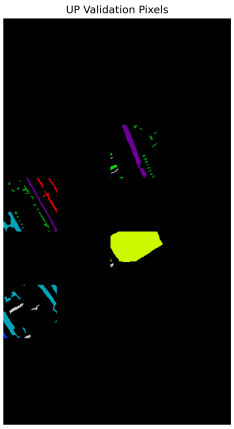
(d) UP Block 1x1: testing



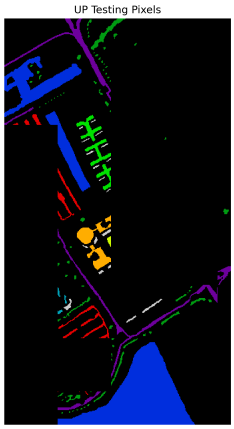
(e) UP Block 80x80: overview



(f) UP Block 80x80: training



(g) UP Block 80x80: validation



(h) UP Block 80x80: testing

Figure 1: Comparison between the 1×1-pixel control split and the 80×80-pixel spatial block split for the University of Pavia dataset.

score obtained using the largest block size. The differences were then averaged across all three datasets. Under the block split setting, HybridSN showed a substantial reduction in performance, with average decreases of 19.95 in OA, 17.71 in AA, and 24.51 in Kappa. Further details are provided in Table II.

In a random pixel split, training and test samples usually spatially close to each other, allowing the model to benefit from the spatial similarity and distribution of nearby pixels. As a result, the random split protocol can produce optimistic performance estimates. In contrast, the block split setting separates the data spatially, making the test set less similar to the training set and providing a stricter evaluation of generalization ability.

The performance reduction may reflect both spatial distribution shift and reduced class-wise training sample. Figure 1f shows that some classes have substantially fewer training samples than others. Although the procedure ensures that every class is represented in each split, it does not guarantee a sufficient number of training samples per class. With larger

blocks, spatially concentrated classes may contribute fewer samples to the training set.

E. Train-Only PCA with Block Split

Another potential source of data leakage is in the PCA step. In the conventional protocol, PCA is fitted using the entire dataset before the train, validation, and test splits are created. As a result, information from the validation and test sets may indirectly influence the PCA transformation applied during training.

To address this, PCA was fitted using only the training data. The resulting transformation was then applied to the validation and test sets. Under this setting, HybridSN maintained strong overall performance, suggesting that the model is not highly dependent on information introduced through dataset-wide PCA fitting.

The train-only PCA protocol was also evaluated in combination with the block split setting. Compared with the baseline results, the average performance reduction was 18.46 in OA, 16.88 in AA, and 22.76 in Kappa, more detail is shown in Table IV. This represents an average improvement of 1.49 in OA, 0.83 in AA, and 1.75 in Kappa relative to the block split protocol, as illustrated in Figure 2.

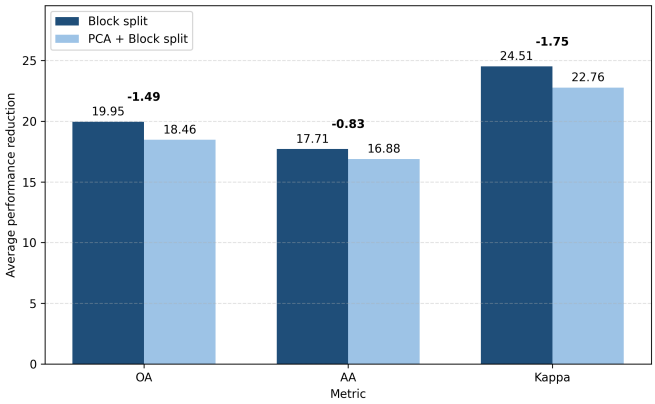


Figure 2: Performance improvement of train-only PCA with block split compared with the block split setting

V. Future Work

Table III presents an anomalous result observed under seed 9 on the IP dataset. Both the PyTorch reimplementation and the reproduction produced unusually low performance scores. Since the PyTorch reimplementation used 30 PCA components while the reproduced implementation used 50 PCA components, this result should be interpreted as a repeated anomalous pattern rather than a fully controlled comparison. Investigating the cause behind this behavior could be a future work for HybridSN.

Future work should report and account for class-wise training sample counts. This would help distinguish the effects of spatial separation from those of insufficient training data for individual classes.

The Figure 2 shows that fitting PCA only on the training data produced a modest improvement under the block split setting. Additional analysis is required to determine whether this improvement is related to better use of spectral information or other factors.

VI. Conclusion

This study reproduced and evaluated HybridSN under several experimental settings. The baseline results show that the reproduced implementation achieved performance broadly comparable to the original paper and PyTorch reimplementation results, supporting the reproducibility of HybridSN.

The experiments also show that evaluation protocol has a strong effect on the reported performance. Validation-based checkpoint selection is important to avoid overfitting, while block split evaluation provides a stricter approximation of spatial generalization than random pixel splitting. However, because patch-based models can still share neighbouring pixels across split boundaries, block splitting should be interpreted as leakage-reduced rather than fully leakage-free unless an additional buffer is used.

Overall, this work highlights the reproducibility of HybridSN and the importance of careful experimental design in hyperspectral image classification. The availability of the original HybridSN code and configuration made this reproducibility analysis possible.

References

[1] S. K. Roy, G. Krishna, S. R. Dubey, and B. B. Chaudhuri, "HybridSN: Exploring 3-D–2-D CNN feature hierarchy for hyperspectral image classification," *IEEE Geoscience and Remote Sensing Letters*, vol. 17, no. 2, pp. 277–281, 2020.

[2] J. Liang, J. Zhou, Y. Qian, L. Wen, X. Bai, and Y. Gao, "On the sampling strategy for evaluation of spectral–spatial methods in hyperspectral image classification," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 55, no. 2, pp. 862–880, 2017.

[3] M. Ahmad, M. Mazzara, S. Distefano, A. M. Khan, and H. A. Altuwaijri, "Improving generalization for hyperspectral image classification: The impact of disjoint sampling on deep models," *Computers, Materials & Continua*, vol. 81, no. 1, pp. 503–532, 2024.

[4] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, p. 100804, 2023.